Task Management API - Stage 2: Read Endpoints & 404 Defensive Handling

Overview

This repository houses a lightweight RESTful Task Management API built using Node.js and Express in VS Code on Windows as part of the FlyRank Internship Backend Engineering Track (Week 2).

Stage 2 establishes read capabilities (GET /tasks and GET /tasks/:id) operating over an in-memory database cache, enforcing strict HTTP 404 status codes and structured error payloads for non-existent records.

Technical Stack & Environment

- **Runtime:** Node.js (v24.x)
- **Framework:** Express.js
- **Data Layer:** In-Memory Volatile Array (RAM Cache)
- **Environment:** VS Code Integrated Terminal (Windows PowerShell)
- **Verification Tool:** curl.exe (Native Windows Binary)
- **Version Control:** Git

API Catalog & Route Architecture

Method	Endpoint	HTTP Success	HTTP Error	Sysadmin / IT Analogy
GET	/	200 OK	—	IIS Web Site Root Banner / Discovery
GET	/health	200 OK	—	Load Balancer ICMP Heartbeat Ping
GET	/tasks	200 OK	—	Querying service list (Get-Service)
GET	/tasks/:id	200 OK	404 Not Found	Querying Event Viewer

				ID (Get-WinEvent -InstanceId)

Data Schema (tasks array)

Each record in memory strictly adheres to the following JSON schema:

```
{
  "id": 1,
  "title": "Responding to routine client emails",
  "done": false
}
```

Seeded In-Memory State:

1. id: 1 — "Responding to routine client emails" (done: false)
2. id: 2 — "Monitoring server status alerts" (done: false)
3. id: 3 — "Prepare for client meeting" (done: false)

Service Lifecycle & Execution

1. Launch the Server Daemon

In your primary (left) VS Code PowerShell terminal tab:

```
node server.js
```

Expected Startup Output:

```
Task API Daemon running on http://localhost:3000
```

Verification Protocol (curl.exe Inspections)

Run these tests in your secondary (right) VS Code PowerShell terminal tab using curl.exe to display HTTP response headers (-i).

1. Retrieve All Records (GET /tasks)

```
curl.exe -i http://localhost:3000/tasks
```

- **Expected Status:** HTTP/1.1 200 OK
- **Response:** Array containing all 3 seeded task objects.

2. Retrieve Specific Record (GET /tasks/1)

curl.exe -i http://localhost:3000/tasks/1

- **Expected Status:** HTTP/1.1 200 OK
- **Response Body:**

{"id":1,"title":"Responding to routine client emails","done":false}

3. Retrieve Non-Existent Record (GET /tasks/99)

curl.exe -i http://localhost:3000/tasks/99

- **Expected Status:** HTTP/1.1 404 Not Found
- **Response Body:**

{"error":"Task 99 not found"}

Defensive Engineering Logic

1. **Path Parameter Conversion:** Route parameters captured from the URL (req.params.id) arrive as string primitives and are safely converted using parseInt(req.params.id, 10).
2. **Explicit 404 Firewall:** If .find() fails to locate a matching primary key, Express intercepts the request and terminates execution early with res.status(404).json(...).
3. **Machine Readability:** Downstream client systems, mobile applications, and automated monitoring bots rely on standard status codes (404) rather than empty 200 OK responses to determine operation status.

Version Control Checkpoint

Save all changes and verify your Git audit trail:

```
git add .
git commit -m "stage2: read endpoints with 404"
git log --oneline
```